StreamVault performance What we learned

Read this first: measure the failing resource, change one thing, repeat the same TV scenario.

Session: November 4, 2026 | Sony BRAVIA 4K VH2 | Android 12 | Manual device verification; no automated tests run.

The three problems

1. Playback crashed. Video slowed down, the app died, and the library loaded again.
2. Cold startup was expensive. Downloaded files were cached, but parsing and indexing ran again.
3. Organized Movies was slow. Switching from Series to Movies took about a minute.

Measured outcomes

Measure	Before	After
Loaded Java allocations, organization off	~150 MiB	~137 MiB
Playback on the checked stream	OOM / restart	~152 MiB; stable manual session
Base library processing	104.37 s from sources	4.58 s from snapshot
First Series catalog preparation	13.34 s	5.25 s after parsing changes
First Movies catalog preparation	56.45 s	29.14 s after parsing changes
Series / Movies after cold app restart	Recompute catalog	0.16 s / 0.65 s disk restore

These are device observations, not guarantees. Cache restore timings exclude base-library startup and final rendering. A full TV restart felt about 15 s to the user. Title counts matched: 10,950 series titles and 56,442 movie titles.

Reading map

2 Crash diagnosis | 3 Library memory | 4 Architecture
5 Startup caching | 6 Organization | 7 Debugging playbook
8 Interview explanation and code map

Playback crash: find the limit

The fatal stack shows where an allocation failed. It does not necessarily show who consumed the memory.

Problem and symptoms

Playback slowed down after a few seconds. The interface stalled, audio buffers ran short, the app crashed, and a fresh process loaded the library again.

Evidence we gathered

Captured **logcat** over network ADB, then filtered the saved file locally for fatal exceptions, out-of-memory errors and decoder failures. Historical Sony/system-service crashes were separated from **com.streamvault.tv** failures.

Decisive evidence: Java heap at 191 MiB of a 192 MiB growth limit; repeated OutOfMemoryError; collections freeing almost nothing; pauses lasting seconds and 218 skipped frames.

The final failure occurred in SharedPreferences XML serialization. Earlier failures also occurred in the stream loader and MediaCodec input handling. Several unrelated allocations failing together pointed to **process-wide Java memory exhaustion**. It did not prove a decoder-format problem.

First fix: reduce playback buffering

PlayerController now configures DefaultLoadControl with a **12 MiB byte target**, a 5-15 s buffering window, and 1 s / 2 s start / rebuffer thresholds. Size takes priority over the time target.

The byte target is not a cap on the whole player or app. Decoder, audio, network and native allocations are separate. Smaller buffers can increase rebuffering on an unstable connection.

How we verified

Installed a signed release update over ADB without clearing data. Replayed the same stream and sampled memory. With the buffer change, playback lasted over 10 minutes, the process ID stayed the same, allocations settled near 165 MiB, and checked logs showed no new OOM. Later library fixes reduced playback allocations to about 152 MiB.

A stable session validates this scenario; it does not prove every codec, bitrate or browsing path is safe.

Library memory: remove real waste

Profile the loaded baseline before blaming playback. This library was already using about 150 MiB before video started.

How we investigated

Compared dumpsys meminfo before and during playback. One report timed out during severe stalls; that timeout was not a memory measurement. A release heap capture was blocked, so we used a temporary debuggable build, kept the heap local, and inspected class counts and field sizes.

Debugging disabled R8 code optimization. Its heap was useful for finding object types, not for comparing absolute totals with the release. Heap estimates were shallow-size estimates, not a full retained-size analysis.

Two focused fixes

Finding	Change	Release observation
The URL index retained ordering links and excess capacity. A separate list already held browse order.	LinkedHashMap to HashMap; right-sized capacity; separate ordered list preserves first URL occurrence.	~150 to ~147 MiB allocated
About 423,000 Channel objects; tvgId strings on nearly every entry. Source search found no reader of that field.	Removed tvgId from Channel and parser output. Stream URLs still identify playback, progress and favorites.	~147 to ~137 MiB allocated

Why it mattered

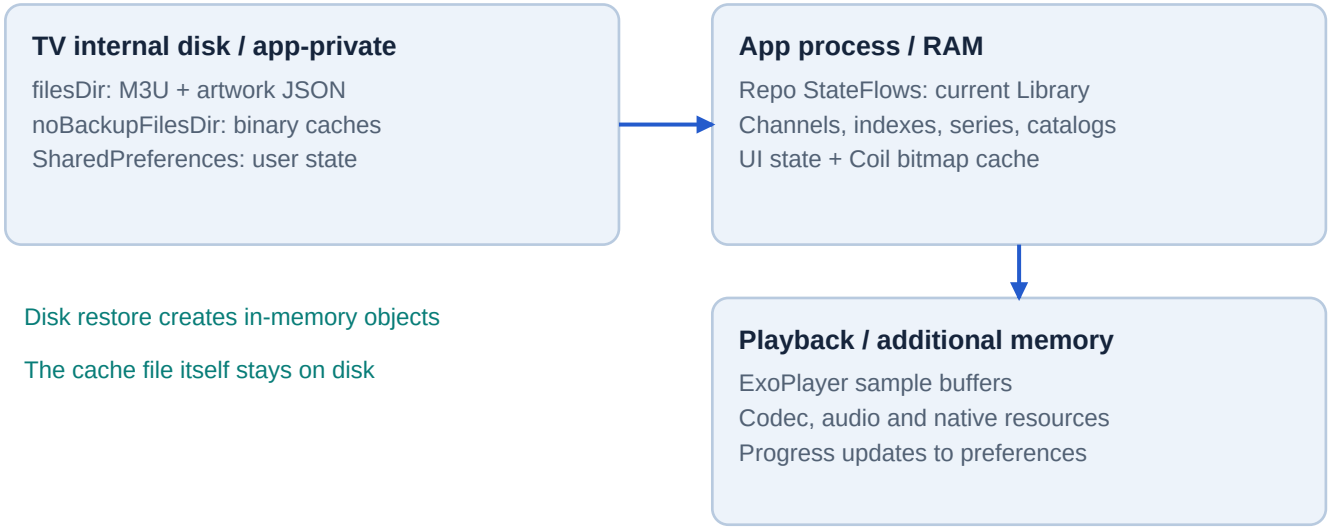
A small unused field becomes expensive when repeated hundreds of thousands of times. The capture estimated roughly 20 MiB for its strings/backing arrays in the diagnostic build. The measured release saving was closer to 10 MiB. We reported the measured result, not the estimate as a guarantee.

Verification and headroom

The normal release was restored. The loaded baseline dropped by about **13 MiB in total**. Playback later settled near **152 MiB**, leaving around **40 MiB** below the observed Java limit. No automated tests were run; builds and manual TV checks supplied the evidence.

Where the data lives

Disk storage survives a restart. RAM holds the objects the running app currently needs.



RAM relationships

Library.all preserves browse order. **byId** maps a URL to the same Channel object. Group lists and Series episode lists also reference those objects. Indexes add overhead, but references do not duplicate every Channel.

Organized catalogs hold cleaned titles and versions that reference existing Channel or Series objects. Shared metadata and artwork reduce repeated strings and objects. Artwork images decoded for display have their own memory cost.

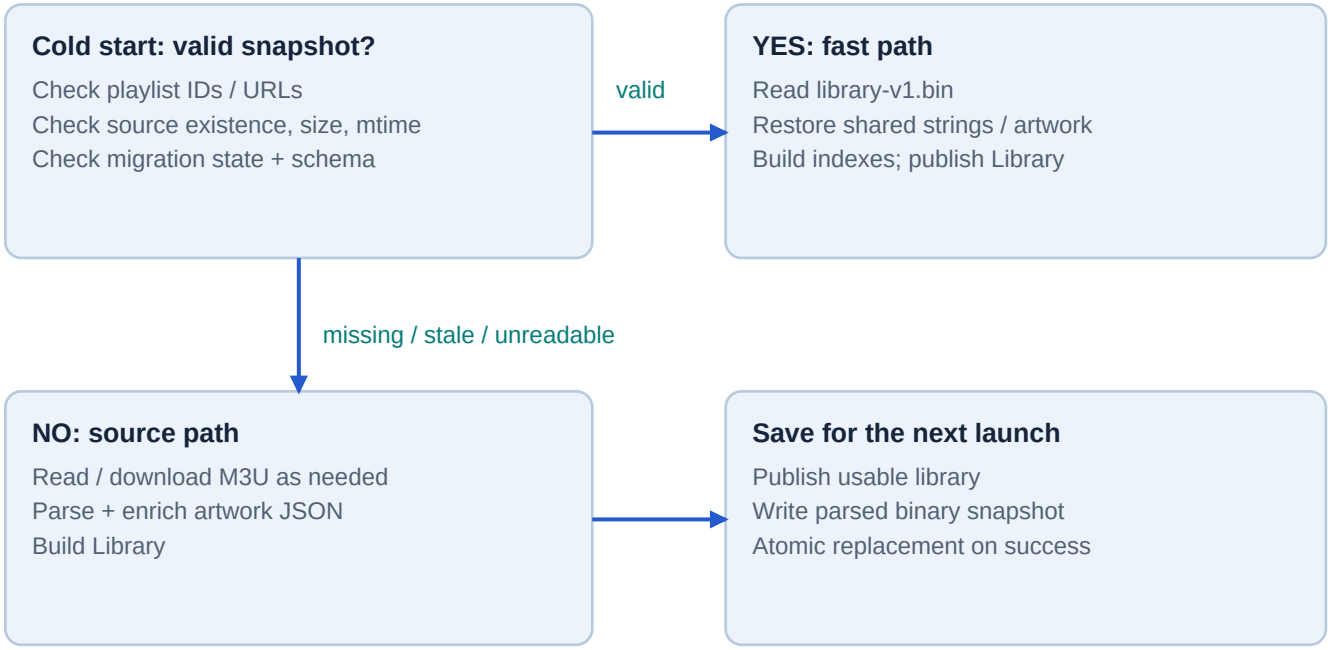
Read memory numbers correctly

Metric	Meaning in this investigation
Dalvik Heap / Heap Alloc	Allocated Java objects. We used this for the 137 / 152 / 154 MiB comparisons.
192 MiB growth limit	Observed Java heap limit for this TV/app configuration; not the TV's total RAM.
Total PSS	RAM footprint with proportional accounting for shared pages. Latest organized browsing sample: ~217 MiB.

Latest organized browsing allocations were ~154 MiB, leaving ~38 MiB of Java headroom. PSS and allocated heap measure different things: do not add them together. Native allocations do not share the Java heap limit, but still consume system resources.

Startup: cache parsed data

Caching the download was not enough. Parsing 418,360 unique streams and enriching metadata still took 104 seconds.



Implementation

LibrarySnapshot writes a versioned binary stream in **noBackupFilesDir**. A bounded string dictionary and shared artwork references keep restoration compact. URLs and names stream through without a huge serialization buffer. AtomicFile protects replacement of a working snapshot.

A SHA-256 fingerprint represents playlist configuration and source-file metadata. It is **not a content hash of every source byte**. Missing, stale or unreadable snapshots fall back to the existing loader. A failed source load does not create a supposedly complete snapshot.

Verification

First source load: **104.37 s**. After saving and restarting the app: **4.58 s** to restore the same 418,360 entries - about **23x faster**. Java allocations stayed near **137 MiB**. The user checked categories, posters and watch progress, then restarted the TV and observed about **15 s total launch time**.

Refresh changes source files and invalidates caches. There is no new automatic provider freshness policy here: an unchanged local snapshot does not detect new remote titles by itself.

Organization: measure, then cache

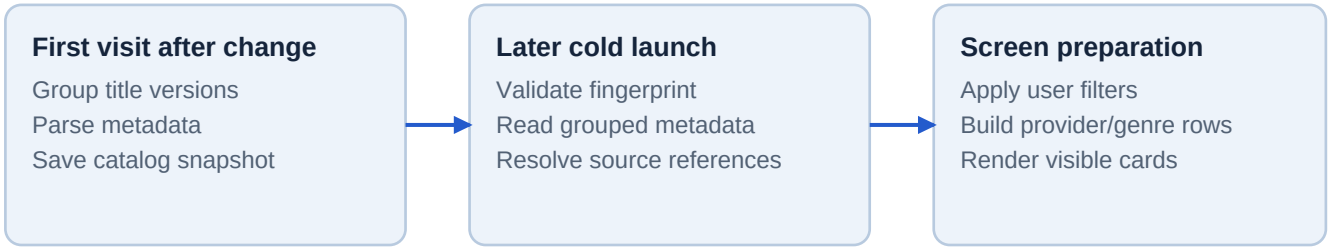
A quick screen can still wait on a slow data pipeline. Rows were fast; catalog preparation was the bottleneck.

Measured stage	Series	Movies
Original catalog total	13.34 s	56.45 s
Original metadata parsing	8.99 s	28.07 s
Original title-key work	2.49 s	17.07 s
Original screen-row preparation	0.11 s	0.16 s
After parsing changes, first build	5.25 s	29.14 s
After persistence, disk restore	0.16 s	0.65 s

Fix 1: avoid repeated calculations

Bounded caches reuse canonical genre labels, genre combinations and group genre results. The combination key excludes each title's year/rating. Grouping cleans a title once and reuses its already-parsed year and cleaned display name. Counts remained 10,950 series titles and 56,442 movie titles.

Fix 2: persist the organized catalog



CatalogSnapshot saves one file per content type. It stores cleaned titles and metadata; version URLs/series keys reconnect to the current Library. It does not duplicate full Channel objects. Fingerprint/schema mismatch or read failure triggers a rebuild. Current language, quality and sort filters still apply after restoration.

Cold app restart confirmed the disk-cache path. Including row preparation, each catalog took under one second after base-library loading. Refresh requires one rebuild. The first-build figures and cache-restore figures describe different paths.

Repeatable debugging playbook

Use the same stream, device and screen state. Record one baseline before changing code.

1. Capture the failure

Connect ADB. Record logcat through the crash and restart. Filter locally, keeping context around matches. Confirm the package and timestamp; historical system crashes are noise.

2. Classify the failure

OOM: inspect heap and allocations. Decoder error: inspect format/capabilities. ANR: inspect blocked threads. Network error: inspect request/connection evidence. Do not assume all playback failures share a cause.

3. Measure the baseline

Sample meminfo after full library load, at playback start, and several minutes later. Compare Heap Alloc with the observed growth limit. A timed-out report is missing data, not zero usage.

4. Profile the expensive path

For memory, inspect heap object counts and field costs. For delays, time parsing, grouping and row preparation separately. Keep heap captures local because they may contain credentials.

5. Change one mechanism

Use a focused patch: buffer budget, smaller index, unused field removal or parsed-data cache. Avoid masking memory growth by merely requesting a bigger heap.

6. Verify the same scenario

Build a release, update without clearing data, repeat playback and tab switches. Check process ID, errors, memory trend, result counts and visible UX. Compare app restart and TV restart separately.

Useful ADB commands

Replace TV_IP with the current address. Run the commands from a terminal with adb on PATH.

```
adb connect TV_IP:5555
adb -s TV_IP:5555 logcat -v threadtime > crash.txt
adb -s TV_IP:5555 shell dumpsys meminfo com.streamvault.tv
```

To isolate timings: adb logcat -d -s CatalogTiming:I LibraryStartup:I "*:S". For a heap: adb shell am dumpheap com.streamvault.tv /data/local/tmp/baseline.hprof (requires a debuggable app).

Explain it in an interview

Lead with the user problem, give the evidence, then the measured improvement and its limits.

A 60-second answer

"An Android TV app crashed during playback. Logs confirmed Java OOM at a 192 MiB limit. I reduced buffering, profiled the library, and removed an unused field repeated across roughly 423,000 channel objects. Library allocations fell from 150 to 137 MiB; checked playback settled near 152 MiB without another OOM.

Startup was reprocessing cached source files. A parsed-data snapshot reduced measured library loading from 104 to 4.6 seconds. Stage timings then exposed expensive movie metadata preparation. Reusing normalization and persisting organized catalogs reduced subsequent Movies catalog restore from a 56.5-second original build to 0.65 seconds. I verified counts, UI, process continuity and memory on the actual TV."

Tradeoffs worth mentioning

Buffering: less memory can mean more rebuffering.
Caching: faster reads require disk space, invalidation and schema versions.
Persistence: the first rebuild is still expensive after source changes.
Evidence: manual observations on one TV/stream are not universal guarantees.

Where to look in the code

File under app/src/main/java/com/streamvault/tv/	Responsibility
player/PlayerController.kt	Buffer budget; progress
data/M3uParser.kt + data/Models.kt	Channel model; lookup index
data/Repo.kt	Startup; source fingerprint
data/LibrarySnapshot.kt	Parsed-library persistence
data/LibraryOrganization.kt	Grouping; normalization; timing
data/CatalogSnapshot.kt	Organized catalog persistence
ui/OrganizedLibrary.kt	Filters and row timing

Evidence: local-test-data/ contains diagnostic logs, meminfo reports and the baseline heap. Later ADB timing and memory readings were observed during this session; not every reading was saved as a separate file. No credentials or playlist contents are reproduced here. Final measured build: 1.0.11. Earlier signed builds provided the step-by-step comparisons.